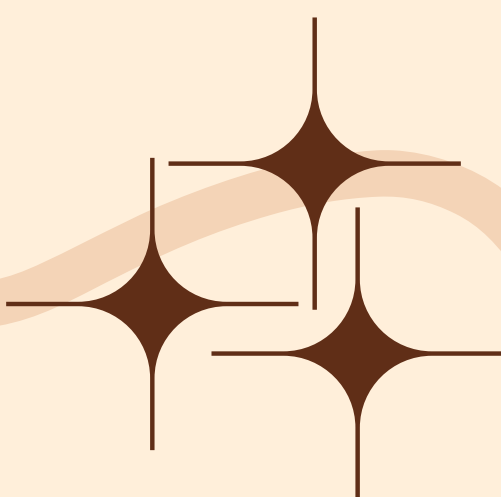




DISCORD & SPOTIFY



Nomes: Felipe Godinho Dal Molin e Vinícius Giroti
Matéria: Redes de computadores
Professor: Charles Christian Miers

SUMÁRIO

Discord

1

Agentes envolvidos

2

Histórico das principais versões

3

Principais elementos da versão mais recente

4

Peer-to-Peer

5

Cliente-Servidor

6

Protocolos de aplicação

13

Versões dos protocolos utilizados

Spotify

7

Agentes envolvidos

8

Histórico das principais versões

9

Principais elementos da versão mais recente

10

Peer-to-Peer

11

Cliente-Servidor

12

Protocolos de aplicação

The background is a light beige color with decorative elements. In the top left, there is a dark brown wavy line and a small floral motif. In the top right, there is a dark brown wavy line and a branch with several leaves. In the bottom left, there is a dark brown wavy line and a small floral motif. In the bottom right, there is a dark brown wavy line and a small floral motif.

DISCORD

AGENTES ENVOLVIDOS

- Criado por Jason Citron e Stanislav Vishnevskiy em 2015
- Hammer & Chisel, hoje, Discord Inc.
- Contexto:
 - Ausência de aplicativos confiáveis para jogar conversando com os amigos
 - Recursos de comunicação do jogo “Fates Forever”, jogo desenvolvido pela Hammer & Chisel e aclamado pela crítica
- Surge então, em maio de 2015 o aplicativo “Discord”
- Negociação não finalizada com a Microsoft
- 100 milhões de dólares investidos pela Sony
- Segue independente



Jason Citron



Stanislav Vishnevskiy

AGENTES ENVOLVIDOS

- **Contexto 1:**

- Março de 2020, início da pandemia, começa a corrida por plataformas de áudio para o trabalho remoto. Zoom, Microsoft Teams, Google Meet apresentam cada uma problema diferente. Enquanto isso o Discord funcionava muito bem, até com transmissão de vídeo em tempo real.
- Abordagem tradicional custosa que o Zoom realizava: envio da voz para os servidores do Zoom e a distribuição para os outros participantes da chamada.
- Abordagem do Discord: um sistema híbrido com auxílio do servidor central, que se adapta conforme às condições da rede.



Jason Citron



Stanislav Vishnevskiy

AGENTES ENVOLVIDOS

- **Contexto 2:**

- Em 2017, os banco de dados “cassandra-messages” possuíam 12 nós do Cassandra, armazenando bilhões de mensagens.
- No início de 2022, ele tinha 177 nós com trilhões de mensagens.
- Notificações constantes ao time de plantão sobre problemas com o banco de dados, latência imprevisível e custos de operações de manutenção elevadas.



Jason Citron



Stanislav Vishnevskiy

HISTÓRICO DAS PRINCIPAIS VERSÕES

Ano	Descrição
Maio de 2015	Primeira versão do Discord
Primavera de 2016	Lançamento da sobreposição de jogos e da API oficial do Discord
Verão de 2016	Chamadas de voz nas mensagens diretas e emojis personalizados nos servidores
Início de 2017	Discord Nitro
Outono de 2017	Chamada de vídeo e compartilhamento de tela nas mensagens diretas e grupos privados e o lançamento do Rich Presence

HISTÓRICO DAS PRINCIPAIS VERSÕES

Ano	Descrição
Verão de 2019	Lançamento do Go Live (Transmitir jogos ao vivo no servidor) e dos impulsos ao servidor
Primavera de 2021	Vinculação de contas com o Playstation
Primavera de 2023	Bate-papo de voz no Playstation, Soundboard e mensagens de voz em dispositivos móveis
Primavera de 2024	Lançador de Aplicativos, permitindo as atividades de chamada
Inverno de 2024	Bate-papo de texto e voz diretamente em jogos que o suportam

PRINCIPAIS ELEMENTOS DA VERSÃO MAIS RECENTE

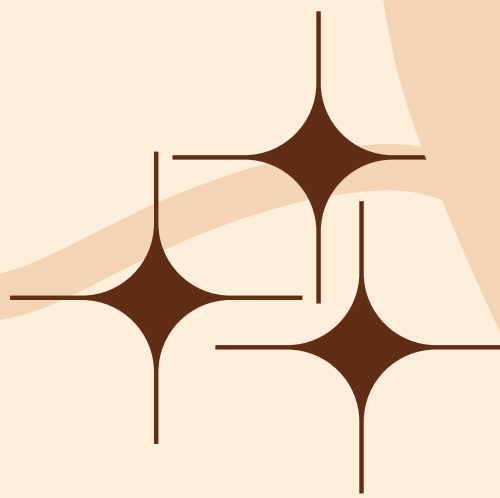
- Chamada de áudio/vídeo
- Bate-papo direto
- Servidores/Guildas
- Canais de texto
- Compartilhamento de tela
- Compartilhamento de arquivos
- Integração com outros aplicativos
- Acesso ao aplicativo sem precisar sair do jogo
- Arquitetura multicamadas
 - “Gateway” inteligente
 - Processamento de áudio multicamada
 - Protocolo de rede em malhas e SFU
 - Controlador de qualidade preditiva
- Filosofia “Gaming-first”
- Movido pela comunidade
- Padrão de uso contínuo

PARTE PEER-TO-PEER

- O Discord implementa o uso do WebRTC, que é um pacote de software desenvolvido pela Google em 2011
- WebRTC:
 - Paradigma padrão: peer-to-peer
 - Utiliza-se do Session Description Protocol (SDP) para trocar informações
 - Secure Real-time Transport Protocol (SRTP) e Datagram Transport Layer Security (DTLS) para codificação
 - APIs para se conectar a um servidor Interactive Connectivity Establishment (ICE) com base nas configurações de ICE de cada peer (funciona como um Tracker server)
 - Determina candidatos com base na qualidade da conexão
 - Caso a conexão seja ruim entre os peers, é preferível um paradigma cliente-servidor
 - Caso a conexão seja boa, uma conexão peer-to-peer é realizada
 - Utilizado com seus padrões no cliente WEB do discord

PARTE PEER-TO-PEER



- Normalmente utilizado no Discord em:
 - Chamadas de voz diretas
 - Compartilhamento de arquivos
 - Canais de voz em servidores com poucos participantes
 - Conexão entre os participantes está boa
- 

PARTE PEER-TO-PEER

- Para o armazenamento das mensagens, o Discord usa o ScyllaDB, que é um database NoSQL peer to peer, que é desenhado para aguentar um volume imenso de informações sem ter nenhum nó falhando, provendo ao sistema uma acessibilidade bem estável.
- No ScyllaDB, o banco de dados é dividido em vários clusters com vários nós que armazenam seus dados, sendo qualquer um deles capaz de responder como um cliente-servidor para alguma requisição, entretanto, a comunicação entre nós é feita no paradigma peer-to-peer.

PARTE CLIENTE-SERVIDOR

- Além do suporte padrão ao paradigma peer-to-peer, o WebRTC também suporta o paradigma Cliente-Servidor
- Uso de WebSockets (Discord Gateways)
- Utiliza uma arquitetura Selective Forwarding Unit (SFU) para encaminhar os dados de áudio e vídeo, o qual também age como uma ponte entre os diferentes protocolos utilizados no Discord de Navegador e Aplicativo e maneja o RTP Control protocol (RTCP)
- Utiliza o UDP na camada de transporte para estabelecer conexão
- Utiliza-se do “Google Cloud Platform” para hospedar mais de 850 servidores em 13 regiões em 30 datacenters diferentes
- Utilizado no Discord:
 - Conexão entre os participantes da chamada está ruim
 - Canal de voz com maior número de pessoas em servidores
 - Compartilhamento de tela em um canal com mais pessoas

PROTOS DE APLICAÇÃO UTILIZADOS

- **SDP**

- **Protocolo de descrição de Sessão**
- **Tem o intuito de adquirir as informações nos canais de mídia**
- **Informar que uma sessão existe e prover informações o suficiente para participar da sessão**
- **Protocolos de aplicação que normalmente são aplicados junto ao SDP, dependendo da situação:**
 - **Session Initiation Protocol (SIP)**
 - **Real-Time Streaming Protocol (RTSP)**
- **Planejado para ter protocolos de transporte específicos para cada situação:**
 - **Transmission Control Protocol (TCP)**
 - **User Datagram Protocol (UDP)**
 - **Real-Time Transport Protocol (RTP), que funciona junto ao TCP ou UDP**
- **Protocolo de aplicação padrão do WebRTC, utilizado pelo Discord**

- **HTTPS**

- **Protocolo é utilizado para o aplicativo Web do Discord**

DIAGRAMA DE SEQUÊNCIA/EXEMPLO DE USO AO SE CONECTAR A UM CANAL DE VOZ

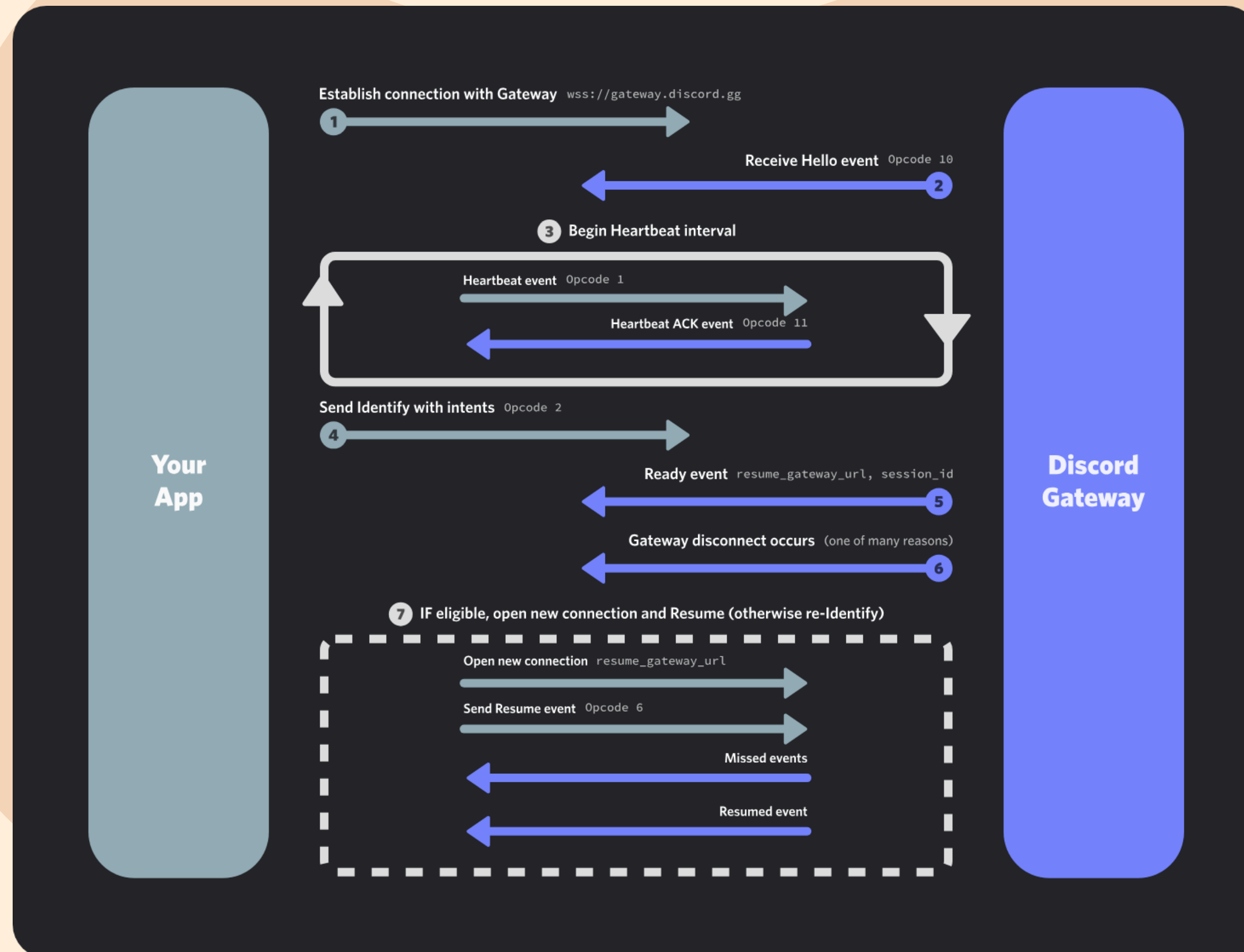
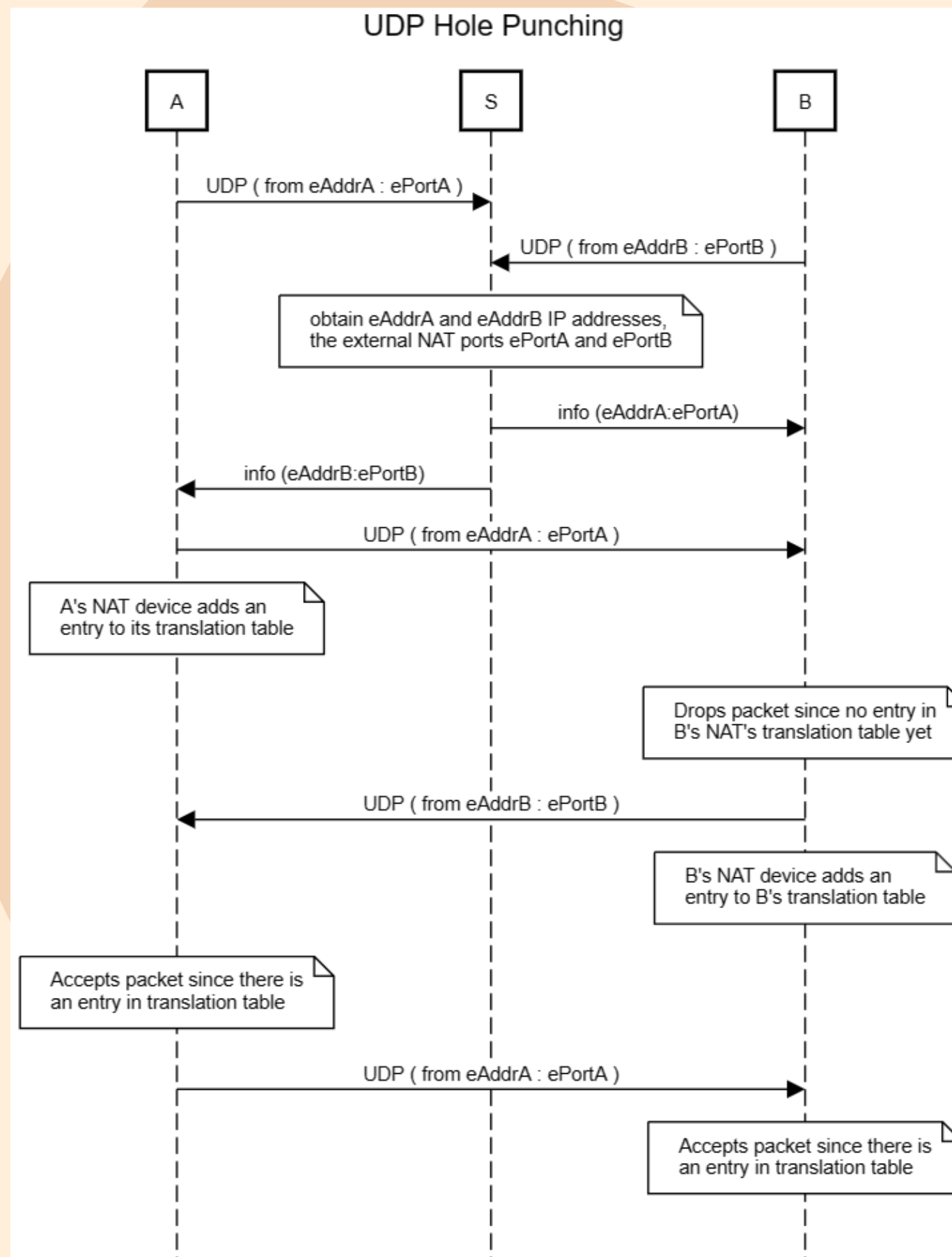


DIAGRAMA DE SEQUÊNCIA/EXEMPLO DE USO AO SE CONECTAR A UM CANAL DE VOZ

- Primeiramente, é necessário descobrir o servidor de áudio para se conectar. Para isso, obtemos o Gateway do Discord usando o GET GATEWAY, e o server responde com o status code 10 “Hello”.
- Para manter essa conexão, continuamos em uma sequência de status code 1 “heartbeats” e status code 11 “heartbeats ACK”.
- Após isso, enviamos status code 2 “identify” para começar o handshake com o gateway. Quando o Discord estiver pronto, ele envia status code 0 (READY) estabelecendo a conexão.

DIAGRAMA DE SEQUÊNCIA/EXEMPLO DE USO AO SE CONECTAR A UM CANAL DE VOZ



- Após receber do Server o status code 0 READY, contendo a informação do servidor de áudio destino, respondemos com status code 1 SELECT PROTOCOL (UDP), iniciando a sequência de UDP Hole Punching.

The background is a light beige color with decorative elements. In the top left, there is a dark brown wavy line and a small floral motif. In the top right, there is a dark brown wavy line and a branch with several leaves. In the bottom left, there is a dark brown wavy line and a small floral motif. In the bottom right, there is a dark brown wavy line and a small floral motif.

SPOTIFY

AGENTES ENVOLVIDOS

- Empresa fundada por Daniel Ek e Martin Lorentzon em 2006
- Aplicativo criado em 2008, pela Spotify AB
- **Motivação:**
 - Combate à pirataria
 - Dificuldade de acesso legalizado para as músicas
 - Crescimento da Internet facilitou a ideia
 - Propostas que beneficiam tanto artistas tanto ouvintes
 - Foco na experiência do usuário
- Artistas podem fazer upload de suas músicas de maneira gratuita
- Investimentos da Founders Fund em 2010 e investimentos de diversos fundos diferentes até 2018, quando entrou no mercado de ações



Daniel Ek



Martin Lorentzon

HISTÓRICO DAS PRINCIPAIS VERSÕES

Ano	Descrição
2008	Primeira versão do Spotify no modelo Freemium
2009	Lançamento do Spotify em celulares e adição da funcionalidade de “escuta offline” para assinantes premium
2010	Introdução de qualidade de áudio superior para assinantes premium
2012	Introdução da funcionalidade “Rádio” para descobrir novas músicas
2013	Adição da playlist semanal “Descobertas da Semana”
2014	Retrospectiva anual + Migração de paradigma híbrido para Cliente-Servidor

HISTÓRICO DAS PRINCIPAIS VERSÕES

Ano	Descrição
2015	Algoritmo de recomendação + Conteúdo de vídeo e podcasts
2016	Lançamento do “Radar de novidades” com lançamentos de artistas que o usuário acompanha.
2018	Artistas agora podem publicar de maneira independente no Spotify
2020	Stories em playlists
2021	Recurso “blend” que permite que 2 usuários combinem o gosto musical em uma playlist
2022	Lançamento de catálogo de audiobooks

HISTÓRICO DAS PRINCIPAIS VERSÕES

Ano	Descrição
2023	Lançamento do “DJ”, ferramenta de IA que controla as recomendações e adiciona comentários às playlists.
2024	“Playlist in bottle”, que funciona como uma cápsula do tempo musical + Interação entre fãs e artistas pelas “Listening Parties”
2025	Spotify “HiFi”, com uma nova qualidade de áudio superior, sem perda de áudio + Nova funcionalidade de “Mensagens” para compartilhamento de conteúdo no App

PRINCIPAIS ELEMENTOS DA VERSÃO MAIS RECENTE

- **Pick & Play:** Escolher e tocar qualquer música no App
- **Search & Play:** Pesquisar uma faixa e tocá-la imediatamente
- **Share & Play:** Escutar músicas compartilhadas por amigos
- Capas de playlists personalizadas
- Tempo entre transições de música personalizado
- Diferentes qualidades de som para as músicas
- **Spotify Connect:** Reproduzir música em outros dispositivos compatíveis
- Regras contra imitação vocal
- Filtragem de spams musicais
- Uso de IA

PARTE PEER-TO-PEER

De acordo com Alison Bonny, Diretor de comunicação da empresa, em 2014, o Spotify estava deixando de ser uma estrutura peer-to-peer e se tornando cliente-servidor. Entretanto, a estrutura funcionava da seguinte maneira:

A estrutura peer-to-peer do Spotify é uma rede desestruturada, a construção dessa rede e a sua manutenção é assistida por trackers. Isso permite que todos os nós participem da rede de modo igualitário.

Um cliente se conecta a um peer somente quando deseja baixar uma música no Spotify que ele vê que aquele peer contém.

É feito caches locais das músicas em que o cliente fez download. Esses caches também são ofertados para os peers em que estão conectados.

Não há um roteamento geral feito na rede, então dois peers que querem trocar dados devem estar diretamente conectados.

Fonte: (G. Kreitz, F. Niemela, 2010)

PARTE PEER-TO-PEER: LOCALIZANDO PEERS

O tracker do Spotify funciona similarmente com o protocolo do BitTorrent. Mantém-se o mapeamento de músicas em peers que reportaram recentemente possuir essa música completa.

O tracker mantém uma lista dos 20 peers mais acessados para cada música em que ele guarda.

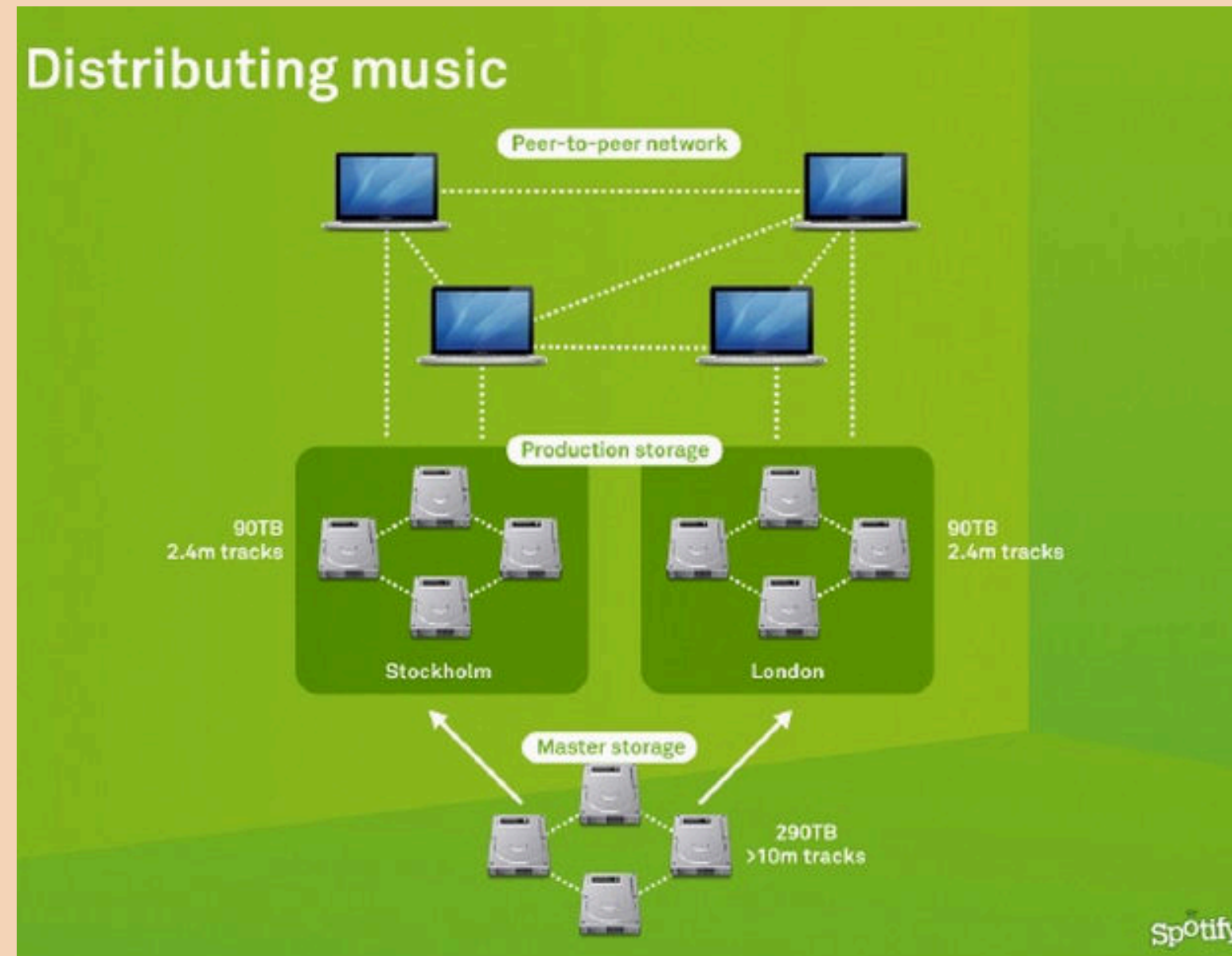
- O cliente mantém uma conexão TCP aberta com o server do Spotify.

Quando um cliente faz uma requisição para saber onde a música poderia estar, é respondido pelo tracker uma lista de até 10 peers que estão online e que possuem essa música.

Também feito uma busca entre os vizinhos dos vizinhos dos peers, dando forward na requisição para todos os vizinhos de um peer.

Fonte: (G. Kreitz, F. Niemela, 2010)

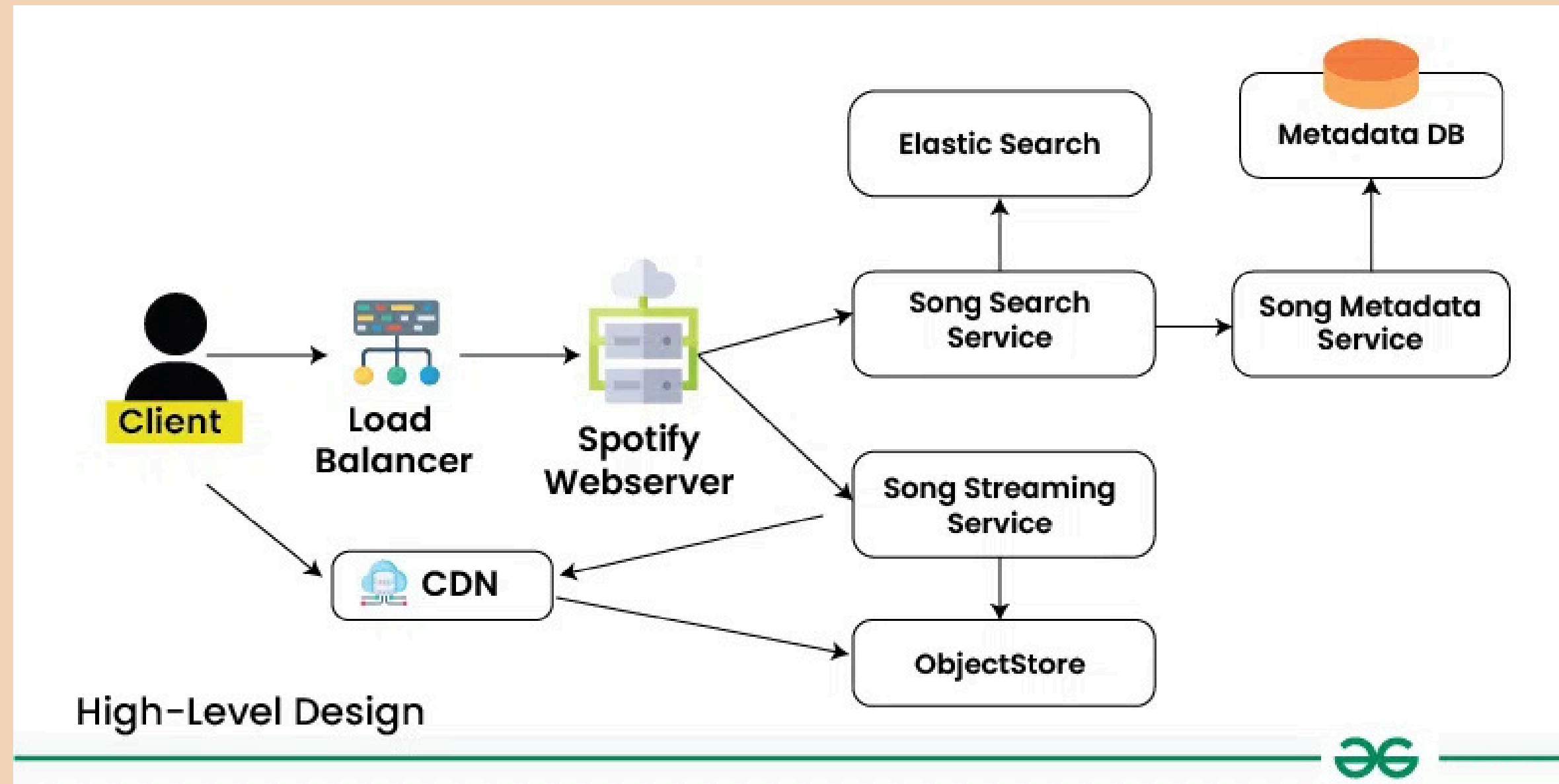
PARTE PEER-TO-PEER



PARTE CLIENTE-SERVIDOR

- Envolve servidores HTTP de conteúdo distribuído globalmente em forma de CDN
- Solicitação do cliente para acessar uma música ou podcast
- Solicitação enviada para a CDN, que envia para o servidor mais próximo ao cliente a requisição, junto às informações do cliente
- Servidor mais próximo envia o conteúdo solicitado pelo cliente em vários pacotes de 512KB, normalmente
- Servidor armazena informações dos interesses do usuário e utiliza isso para a recomendação de novos conteúdos

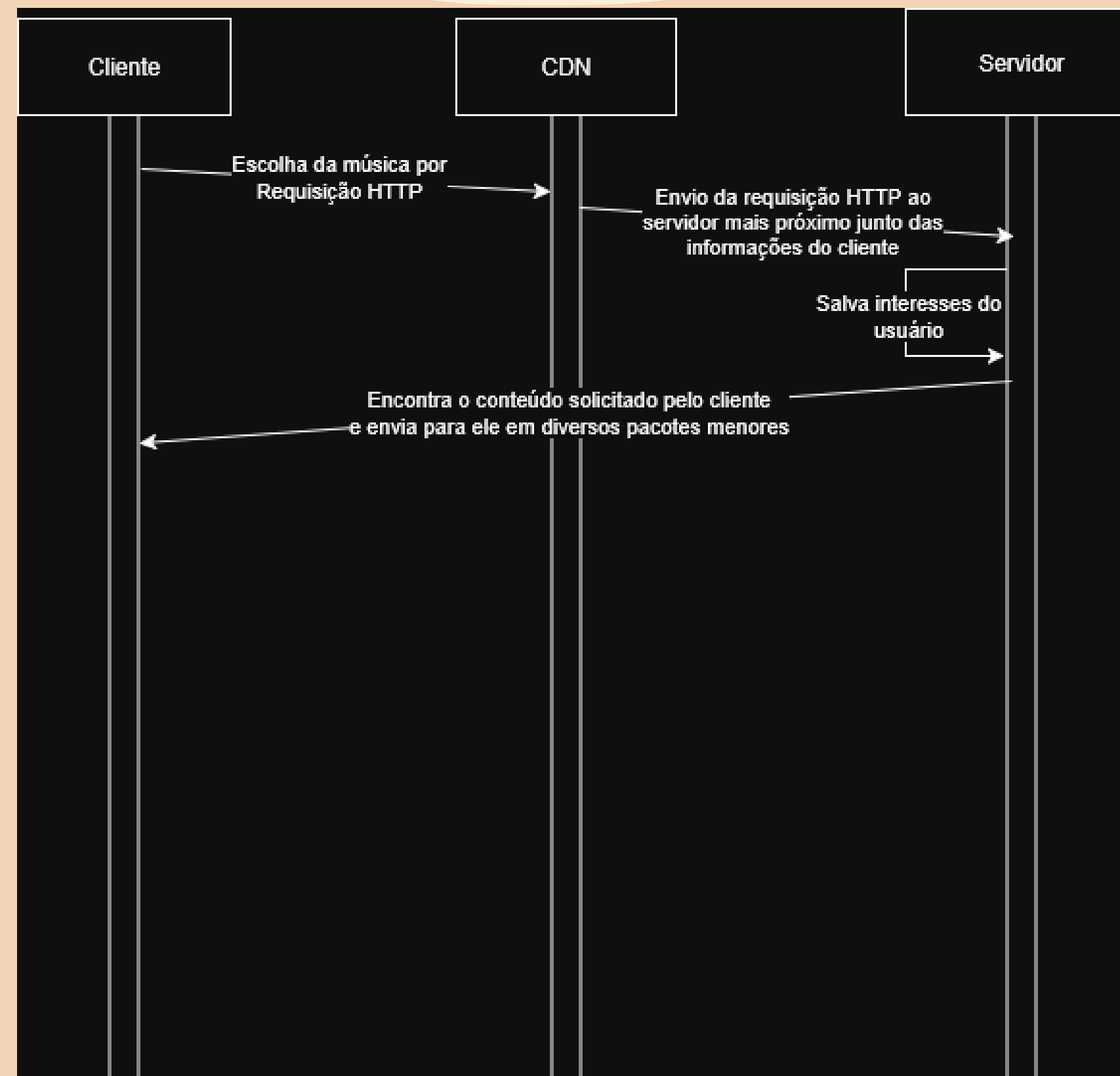
PARTE CLIENTE-SERVIDOR



EXEMPLO DE USO AO DAR PLAY EM UMA MÚSICA

- Cliente loga no aplicativo e escolhe uma música para tocar
- Essa requisição de escolher uma música é interpretada como uma requisição HTTP à rede CDN
- CDN processa a requisição e envia ao servidor mais próximo
- Servidor mais próximo salva interesses e devolve o conteúdo da música solicitado pelo cliente
- Cliente curte a música sem precisar ocupar armazenamento do celular, visto que todo o conteúdo está armazenado apenas no servidor e enviado ao cliente

DIAGRAMA DE SEQUENCIA AO DAR PLAY EM UMA MÚSICA



PROTOCOLOS DE APLICAÇÃO UTILIZADOS

- **HLS**
 - HTTP Live Streaming
 - Aplicado em servidores
 - Divide os arquivos em arquivos menores para download e os envia utilizando o protocolo HTTP
 - Clientes carregam os arquivos HTTP menores e os reproduzem
 - Codificação
 - Segmentação
 - Distribuição
- **RTSP**
 - Real Time Streaming Protocol
 - Controle do fluxo de mídias
 - Pausar
 - Reproduzir
 - Envia os comandos de controle
- **HTTPS**
 - Protocolo é utilizado para o aplicativo Web do Discord

VERSÕES DOS PROTOCOLOS

SDP		
RFC	Ano	Descrição
RFC 2327	1998	Primeira versão, define o formato de texto das informações de mídia, estabelece o tipo representado por uma letra única, introduziu rtpmap e rmap e suportava apenas IPv4.
RFC 4566	2006	Padrão “moderno”. Introduziu suporte ao IPv6, novos atributos para definir os candidatos de um servidor ICE e adicionou camadas de autenticação para quesitos de segurança.
RFC 8866	2021	Versão atual. Incorpora algumas partes do mecanismo ICE, atualizou a terminologia do formato de texto e enfatiza o uso do Secure Real-Time Transport Protocol (SRTP).

VERSÕES DOS PROTOCOLOS

RTP/SRTP		
RFC	Ano	Descrição
RFC 1889	1996	Primeira versão. Definiu o formato do cabeçalho com número de sequência, timestamp e identificador da fonte de transmissão (SSRC) e definiu perfis e formato de payloads.
RFC 3550	2003	Versão atual. Não trouxe grandes modificações. Esclareceu o comportamento de Mixers e Translators, a detecção e tratamento de colisões do SSRC, a sincronização do fluxo, a regra para validação de cabeçalhos e o gerenciamento de membros da seção. Além disso, considera questões de segurança, como autenticação.
RFC 3711	2004	Introdução do SRTP. Introdução da criptografia dos pacotes RCTP. Adição de uma tag de autenticação para identificar fontes legítimas e proteção contra ataques de Replay.

VERSÕES DOS PROTOCOLOS

RTCP/SRTCP		
RFC	Ano	Descrição
RFC 1889	1996	Primeira versão. Definido para monitorar a entrega de dados e fornecer informações de controle. Define o Sender Report (SR), Receiver Report (RR), Source Description Items (SDS), BYE e APP.
RFC 3550	2003	Não trouxe grandes modificações. Melhorou o algoritmo de temporização do envio de relatórios RCTP, solucionando “spikes” na rede quando os múltiplos participantes de uma sessão fossem enviar relatórios.
RFC 3711	2004	Introdução do SRCTP. Introdução da criptografia dos payloads dos pacotes RTP, das tags de autenticação e proteção contra o replay protection com base em um registro os números de sequência já recebidos.

VERSÕES DOS PROTOCOLOS

STUN		
RFC	Ano	Descrição
RFC 3489	2003	Primeira versão. Objetivo de descobrir o endereço de IP e portas públicas, por meio do “binding request” e classificar o tipo de Network Address Translators (NAT).
RFC 5389	2008	Segunda versão. Abandono do mecanismo de classificação do NAT e se tornou componente do framework ICE para coletar os candidatos ICE. Ademais, introduziu um mecanismo de autenticação das mensagens
RFC 8489	2020	Introdução do SRCTP. Introdução da criptografia dos payloads dos pacotes RTP, das tags de autenticação e proteção contra o replay protection com base em um registro os números de sequência já recebidos.

VERSÕES DOS PROTOCOLOS

TURN		
RFC	Ano	Descrição
RFC 5766	2010	Primeira versão. Objetivo de descobrir o endereço de IP e uma porta em servidores de retransmissão , tentando retransmitir os pacotes UDP. Introduz mecanismos de Alocação, Permissão e Canais.
RFC 6062	2010	Adiciona suporte ao TCP
RFC 8016	2016	Mobility With TURN. Manter a mesma alocação ao servidor TURN quando o cliente muda de tipo de rede.
RFC 8656	2020	Reune as extensões de UDP e TCP em um único padrão. Implementa STUN junto do TURN. Define uma consistência nos métodos entre STUN e TURN. Garante um mecanismo de autenticação unificado entre STUN e TURN e gera uma sincronização dos códigos de erros entre os 2 protocolos.

VERSÕES DOS PROTOCOLOS

HLS		
RFC	Ano	Descrição
RFC 8216	2016	Define o HLS. Permite uma taxa de bits variável para os vídeos. Utiliza arquivos de texto no formato .m3u8 para identificação. Baseado em HTTP. Segmenta os arquivos de áudio/vídeo para distribuí-los em pacotes menores.

VERSÕES DOS PROTOCOLOS

RTSP		
RFC	Ano	Descrição
RFC 2326	1998	Primeira versão. Objetivo de gerenciamento da entrega de dados com propriedades de tempo real. Não envia dados, controla a mídia entre cliente e servidor. Utiliza uma sintaxe semelhante ao HTTP. Negociação de parâmetros da transmissão cliente-servidor. Operação sobre diferentes protocolos de transporte
RFC 6062	2016	Pipelining obrigatório de requisições. Introdução do método REDIRECT, permitindo uma melhor distribuição de carga no streaming. Keep-Alive da sessão. Suporte ao IPv6 e obrigatoriedade da codificação UTF-8 para cabeçalhos

REFERÊNCIAS

- BAUGHER, M. et al. The Secure Real-time Transport Protocol (SRTP): RFC 3711. [S. l.]: RFC Editor, 2004. Disponível em: <https://www.rfc-editor.org/info/rfc3711>. Acesso em: 28 set. 2025.
- BEGEN, A. et al. SDP: Session Description Protocol: RFC 8866. [S. l.]: RFC Editor, 2021. Disponível em: <https://www.rfc-editor.org/info/rfc8866>. Acesso em: 28 set. 2025.
- CLOUDFLARE. O que é o HTTP Live Streaming? | HLS Streaming. [S. l.]: Cloudflare, 2025. Disponível em: <https://www.cloudflare.com/pt-br/learning/video/what-is-http-live-streaming/>. Acesso em: 28 set. 2025.

REFERÊNCIAS

- DISCORD. About Discord. [S. l.]: Discord, 2025. Disponível em: <https://discord.com/company>. Acesso em: 27 set. 2025.
- DISCORD. Como o Discord armazena trilhões de mensagens. Disponível em: <https://discord.com/blog/how-discord-stores-trillions-of-messages>. Acesso em: 27 set. 2025.
- DISCORD. Gateway. Disponível em: <https://discord.com/developers/docs/events/gateway>. Acesso em: 28 set. 2025.
- DISCORD. Voice Connections. Disponível em: <https://discord.com/developers/docs/topics/voice-connections>. Acesso em: 28 set. 2025.
- DISCORD. Opcodes and Status Codes. Disponível em: <https://discord.com/developers/docs/topics/opcodes-and-status-codes>. Acesso em: 28 set. 2025.

REFERÊNCIAS

- HANDLEY, M.; JACOBSON, V. SDP: Session Description Protocol: RFC 2327. [S. l.]: RFC Editor, 1998. Disponível em: <https://www.rfc-editor.org/info/rfc2327>. Acesso em: 28 set. 2025.
- HANDLEY, M.; JACOBSON, V.; PERKINS, C. SDP: Session Description Protocol: RFC 4566. [S. l.]: RFC Editor, 2006. Disponível em: <https://www.rfc-editor.org/info/rfc4566>. Acesso em: 28 set. 2025.

REFERÊNCIAS

- KREITZ, Gunnar; NIEMELA, Fredrik. Spotify -- Large Scale, Low Latency, P2P Music-on-Demand Streaming. In: IEEE TENTH INTERNATIONAL CONFERENCE ON PEER-TO-PEER COMPUTING (P2P), 10., 2010, Delft. Proceedings [online]. Piscataway: IEEE, 2010. p. 1-10. Disponível em: <https://ieeexplore.ieee.org/document/5569963>. Acesso em: 28 set. 2025.
- LOPEZ, Marny. Spotify: A deep dive into their tech stack. Devlane Blog, 9 jul. 2025. Disponível em: <https://www.devlane.com/blog/spotify-a-deep-dive-into-their-tech-stack>. Acesso em: 28 set. 2025.

REFERÊNCIAS

- MAHY, R.; MATTHEWS, P.; ROSENBERG, J. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN): RFC 5766. [S. l.]: RFC Editor, 2010. Disponível em: <https://www.rfc-editor.org/info/rfc5766>. Acesso em: 28 set. 2025.
- MULLIGAN, Mark. Evolution of Spotify from music to multi-content platform. Hypebot, 21 jan. 2025. Disponível em: <https://www.hypebot.com/hypebot/2025/01/evolution-of-spotify-from-music-to-multi-content-platform.html>. Acesso em: 28 set. 2025.
- PANTOS, R.; MAY, W. HTTP Live Streaming: RFC 8216. [S. l.]: RFC Editor, 2017. Disponível em: <https://www.rfc-editor.org/info/rfc8216>. Acesso em: 28 set. 2025.
- PERREAULT, S.; ROSENBERG, J. Traversal Using Relays around NAT (TURN) Extensions for TCP Allocations: RFC 6062. [S. l.]: RFC Editor, 2010. Disponível em: <https://www.rfc-editor.org/info/rfc6062>. Acesso em: 28 set. 2025.

REFERÊNCIAS

- REDDY, T. et al. Mobility with Traversal Using Relays around NAT (TURN): RFC 8016. [S. l.]: RFC Editor, 2016. Disponível em: <https://www.rfc-editor.org/info/rfc8016>. Acesso em: 28 set. 2025.
- REDDY, T. et al. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN): RFC 8656. [S. l.]: RFC Editor, 2020. Disponível em: <https://www.rfc-editor.org/info/rfc8656>. Acesso em: 28 set. 2025.
- SAIFI, Sohail. The Genius Architecture Behind Discord's Voice Chat (That Zoom Could Learn From). Medium, 9 jun. 2025. Disponível em: https://medium.com/@sohail_saifi/the-genius-architecture-behind-discords-voice-chat-that-zoom-could-learn-from-1da9a8c5b08f. Acesso em: 28 set. 2025.

REFERÊNCIAS

- SCHULZRINNE, H. et al. RTP: A Transport Protocol for Real-Time Applications: RFC 1889. [S. l.]: RFC Editor, 1996. Disponível em: <https://www.rfc-editor.org/info/rfc1889>. Acesso em: 28 set. 2025.
- SCHULZRINNE, H. et al. RTP: A Transport Protocol for Real-Time Applications: RFC 3550. [S. l.]: RFC Editor, 2003. Disponível em: <https://www.rfc-editor.org/info/rfc3550>. Acesso em: 28 set. 2025.
- SCHULZRINNE, H.; RAO, A.; LANPHIER, R. Real Time Streaming Protocol (RTSP): RFC 2326. [S. l.]: RFC Editor, 1998. Disponível em: <https://www.rfc-editor.org/info/rfc2326>. Acesso em: 28 set. 2025.
- SCHULZRINNE, H. et al. Real-Time Streaming Protocol Version 2.0: RFC 7826. [S. l.]: RFC Editor, 2016. Disponível em: <https://www.rfc-editor.org/info/rfc7826>. Acesso em: 28 set. 2025.

REFERÊNCIAS

- SPOTI TRICK. The Evolution of Spotify: Key Milestones from Launch to Today. Medium, 16 jan. 2024. Disponível em: <https://medium.com/@spotitrack/the-evolution-of-spotify-key-milestones-from-launch-to-today-64e7f1f07c77>. Acesso em: 28 set. 2025.
- SPOTIFY. Celebrating a Decade of Discovery on Spotify. [S. l.]: Spotify, 2018. Disponível em: <https://newsroom.spotify.com/2018-10-10/celebrating-a-decade-of-discovery-on-spotify/>. Acesso em: 28 set. 2025.
- SPOTIFY. ICYMI: Rounding up 2018's New Spotify for Artists Features. [S. l.]: Spotify, 2018. Disponível em: <https://artists.spotify.com/blog/icymi-rounding-up-the-new-spotify-for-artists-features-in-2018>. Acesso em 28 set. 2018.
- TIMELINE OF SPOTIFY. [S. l.]: Timelines, 2024. Disponível em: https://timelines.issarice.com/wiki/Timeline_of_Spotify. Acesso em: 28 set. 2025.

REFERÊNCIAS

- SPOTIFY. Rounding up 2019's Spotify for Artists Updates. [S. l.]: Spotify, 2019. Disponível em: <https://artists.spotify.com/blog/rounding-up-2019s-spotify-for-artists-updates>. Acesso em: 28 set. 2025.
- SPOTIFY. 6 New Features to 'Unwrap' in Your Spotify 2020 Wrapped - Spotify Newsroom. [S. l.]: 2020. Disponível em: <https://newsroom.spotify.com/2020-12-01/6-new-features-to-unwrap-in-your-spotify-2020-wrapped/>
- SPOTIFY. Here's What's in Store for Your 2023 Wrapped - Spotify Newsroom. [S. l.]: Spotify, 2023. Disponível em: <https://newsroom.spotify.com/2023-11-29/wrapped-user-experience-2023>. Acesso em: 28 set. 2025.
- SPOTIFY. Four Must-Use Spotify Features That Bring You Closer to Your Favorite Artists - Spotify Newsroom. [S. l.]: Spotify, 2024. Disponível em: <https://newsroom.spotify.com/2024-09-12/fans-artists-connections-features>. Acesso em: 28 set. 2025.

REFERÊNCIAS

- VASS, Jozsef. How Discord Handles Two and Half Million Concurrent Voice Users using WebRTC. Discord Blog, 10 set. 2018. Disponível em: <https://discord.com/blog/how-discord-handles-two-and-half-million-concurrent-voice-users-using-webrtc>. Acesso em: 28 set. 2025.
- WALLIS, Julian. How Does Spotify Work? Spotify Tech Stack Explored – The Tech Behind Series. Intuji, 1 mar. 2023. Disponível em: <https://intuji.com/how-does-spotify-work-tech-stack-explored/>. Acesso em: 28 set. 2025.
- WEBRTC. Getting started with peer connections. [S. l.]: WebRTC, 2021. Disponível em: <https://webrtc.org/getting-started/peer-connections>. Acesso em: 28 set. 2025.

The background is a light beige color. It features decorative elements: a dark brown wavy line in the top left corner, a dark brown wavy line in the top right corner with a leafy branch extending from it, a dark brown wavy line in the bottom left corner with three starburst shapes, and a dark brown wavy line in the bottom right corner.

OBRIGADO